Part 2: Scaling to Billions of URLs

Architecture Overview

The single-URL crawler from Part 1 needs to be operationalized into a pipeline that can handle a billion URLs a month. The core idea is to decouple crawling from content extraction since they have very different resource requirements — crawlers spend most of their time waiting on network I/O while extractors burn CPU doing NLP. Keeping them as separate services lets us scale and cost each one independently.

```
Input (GCS file or Cloud SQL date-range query)
        |
        ▼
Trigger Service — deduplicate via Bloom filter, normalize URLs, publish to queue
        |
        ▼
Cloud Pub/Sub: url-crawl-topic  (ordering key = domain hash)
        |
        ▼
Crawler Workers (GKE)
  - Check Cloud Memorystore: robots.txt result, JS/static flag, rate limit
  - Fetch via httpx (static) or Playwright (JS-rendered)
  - Route through residential proxy for bot-protected domains
  - If domain rate limit exceeded → extend ack deadline and sleep until
    rate window resets, then process
  - After 5 failures → Dead Letter Topic
  - On success: upload raw HTML to GCS, write status to Redis buffer
  - Compare content hash with previous crawl
  - Push to extraction queue only if content changed
        |
        ▼
Redis Write Buffer → DB Flusher → Cloud SQL (bulk batched writes)
        |
        ▼
Cloud Pub/Sub: url-extract-topic
        |
        ▼
Extraction Workers (GKE)
  - Fetch raw HTML from GCS
  - Parse metadata + run NLP (KeyBERT)
  - Bulk write to Elasticsearch
  - Update stats table via Redis buffer
```

Storage Design

GCS — Raw HTML. All crawled HTML stored at `gs://crawler-raw/{year}/{month}/{domain}/{url_hash}.html.gz`. The URL hash makes every file unique. A crawl date can be added to the path if intra-month versioning is needed. Raw content never lives in a database — GCS is the source of truth, enabling re-extraction without re-crawling. Estimated: 1B URLs × 100KB × 0.1 gzip = ~10TB/month.

Cloud SQL (PostgreSQL) — Operational Tables. Two tables: one for crawl status tracking, one for page metadata. Tables are range-partitioned by `crawl_month` with a B-tree index on `domain` within each partition, so queries like "all Amazon URLs in July" hit only the relevant partition. Writes go through the Redis buffer at this throughput.

Elasticsearch — Extracted Content. Full extracted content, topics, and body text. Chosen because the primary access pattern is document retrieval and full-text search, not analytical aggregation. Self-managed on GKE to avoid the ~5× cost premium of managed Elastic Cloud.

Unified Data Schema

`stats` table (Cloud SQL)

Tracks every URL end-to-end. A UUID generated at ingestion is propagated through every queue, enabling full distributed tracing of any URL through the pipeline.

Column	Type	Description
`uuid`	UUID	Propagated through all queues
`url_hash`	VARCHAR	SHA-256 of normalized URL
`domain`	VARCHAR	
`crawl_month`	DATE	Partition key
`status`	VARCHAR	queued → crawling → crawled → extracted → failed
`error_reason`	VARCHAR	timeout, http_403, robots_disallowed, etc.
`fetcher_used`	VARCHAR	httpx or playwright

Column	Type	Description
`enqueued_at`	TIMESTAMP	
`crawl_completed_at`	TIMESTAMP	
`extracted_at`	TIMESTAMP	

`metadata` table (Cloud SQL)

Stores structured metadata per successful crawl. `content_hash` enables change detection — if the hash matches the previous crawl, the URL is skipped from the extraction queue entirely. Link counts are stored for quick aggregations; full link arrays are stored in GCS alongside the raw HTML to avoid bloating rows at 1B scale.

Column	Type
`url_hash`	VARCHAR (PK)
`content_hash`	VARCHAR
`title` , `description` , `canonical_url` , `lang` , `schema_type` , `page_type`	TEXT/VARCHAR
`internal_link_count` , `external_link_count`	INT
`raw_gcs_path`	TEXT

Elasticsearch document

```
{
  "url_hash": "a1b2c3...", "url": "https://www.rei.com/blog/...",
  "crawl_month": "2025-07", "title": "How to Introduce Your Indoorsy Friend...",
  "body_text": "A campfire, toasted s'mores...",
  "topics": ["friend hike", "outdoor activity"], "topic_scores": [0.60, 0.54],
  "page_type": "article", "description": "...", "h1": [...], "h2": [...]
}
```

Monitoring

- **Cloud Monitoring** — covers GKE pods, Cloud SQL, Memorystore, and Pub/Sub out of the box. Pub/Sub exposes subscription backlog and oldest unacked

message age natively.

- **Stats table** — internal dashboards query the stats table directly to understand throughput, processing times, and failure rates per domain across any time window.
- **Cloud Monitoring dashboards** — infrastructure alerting and operational visibility across all GCP services.

Alert	Condition
Pipeline stalled	Oldest unacked Pub/Sub message > 60 min
High failure rate	`status=failed` > 5% over 15 min
Crawler throughput drop	URLs/sec < 3,000 for > 5 min
Dead Letter spike	Dead Letter Topic > 1,000 messages in 10 min
Extractor backlog	Extraction queue backlog > 10M messages

SLOs and SLAs

The system is designed to sustain **5,000 URLs/second** — capacity for ~13B URLs/month, completing a 1B URL batch in ~2.3 days.

Metric	SLO (Internal)	SLA (External)
Processing throughput	≥ 5,000 URLs/sec	≥ 5,000 URLs/sec
Crawl success rate	≥ 92%	—
End-to-end latency P95	≤ 30 minutes	—
Data freshness	Within 3 days of batch start	—
API uptime	—	99.9% monthly
Raw HTML retention	—	90 days
Metadata retention	—	2 years

Read Path

Point lookups by URL are served from Cloud SQL (metadata table). Topic search and full-text retrieval are served from Elasticsearch. Analytical aggregations — crawl volume by domain, success rates over time — run against the stats table via Cloud SQL read replicas, or BigQuery if query volume grows beyond what Cloud SQL can handle.

Scale and Cost

Workers are stateless and scale horizontally with zero architecture change. Using Little's Law (workers = throughput × latency) with 1s crawl and 3s extraction:

Service	Instance type	Concurrency per instance	Instances at 5,000/sec
Crawler (async IO, httpx)	n2-standard-4 (4 vCPU, 16GB)	~50 concurrent	~100 instances
Extractor (CPU-bound NLP)	n2-standard-4 (4 vCPU, 16GB)	~4 concurrent	~3,750 instances

Extraction needs 37× more instances than crawling — 3× latency difference compounded by a 12× concurrency gap (IO-bound vs CPU-bound). This is the primary reason the two services are kept separate and scaled independently.

At 5,000 URLs/sec a 1B URL batch completes in ~2.3 days. Spot instances are provisioned on-demand and released after the batch finishes — the cost below reflects 2.3 days of runtime, not a full month.

These numbers are based on estimates — we would have to load test the application to get to the actual numbers.

Processing 1B URLs on GCP is estimated at ~$11,000–12,000, with extractor compute (~$8,500) and egress (~$1,200) as the dominant costs. Everything else — storage, messaging, databases, proxy — is relatively minor.

Optimisations

Cost

- **Content hash deduplication** — skip re-extraction when a page hasn't changed since the last crawl. At 20% unchanged rate across 1B URLs, that's 200M fewer NLP jobs per batch — the single biggest cost lever.
- **Batch NLP inference** — extraction workers pull 16–32 messages from Pub/Sub at once and pass them to KeyBERT as a list. One model forward pass for 32 documents instead of 32 individual calls, dramatically improving CPU utilisation per instance.

Reliability

- **Two-queue architecture** — raw HTML is committed to GCS before the URL hash is pushed to the extraction queue. If the extraction service goes down, no data is lost — the queue retains messages and drains when it recovers.
- **Redis write buffer** — decouples workers from Cloud SQL. If Cloud SQL slows down, the buffer absorbs the spike and the DB flusher catches up. Workers are never blocked on the database.

Scale

- **Cache content hashes in Redis** — at high throughput, checking `content_hash` against Cloud SQL for every URL becomes a read bottleneck. Caching in Redis with a 24-hour TTL absorbs almost all lookups and only misses to Cloud SQL on first crawl or cache expiry.
- **Batched writes via Redis → DB flusher** — workers write status updates to a Redis stream; a DB flusher batches them into a single bulk insert to Cloud SQL once per second. Cloud SQL sees one large transaction per second regardless of how many workers are running.